

6. Neural Networks II

4.11.2025

ETH zürich

D MATH

In today's class...

6.6 Convolutional Neural Networks (CNNs)

- The convolution layer
- Detector and pooling layers
- Example: LeNet-5
- CNNs and data priors

6.7 Recurrent Neural Networks (RNNs)

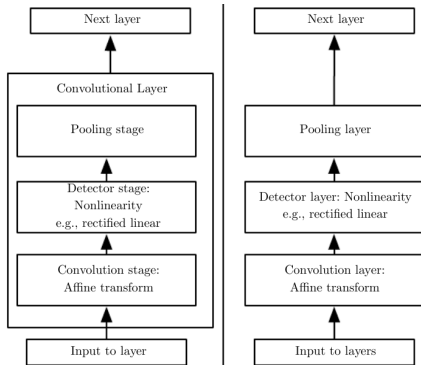
- RNN
- Back-Propagation Through Time (BPTT)
- The vanishing/exploding gradient problem
- Gated networks: GRU and LSTM

6.8 Temporal Convolutional Networks (TCNs)

- TCN architecture
- Residual connections
- Performance

6.6 Convolutional Neural Networks (CNNs)

The basic ingredient of CNNs is the convolutional layer:



- Can be thought of as a layer with three stages (left) or as a composition of three different layers (right).
- Three main components: *convolution layer*, *detector layer* and *pooling layer*.

6.6 CNNs: the convolution layer

The convolution layer implements a matrix operation known as *cross-correlation*.

Definition 1 (Cross-correlation)

Let $I \in \mathbb{R}^{n_1 \times n_2}$ and $K \in \mathbb{R}^{m_1 \times m_2}$ be matrices such that $m_1 \leq n_1$ and $m_2 \leq n_2$, then their cross-correlation is a matrix, denoted by $I \circledast K$, whose entries are given by:

$$(I \circledast K)_{i,j} := \sum_{m=1}^{m_1} \sum_{n=1}^{m_2} I_{m+i-1,n+j-1} K_{m,n}.$$

- Matrix I is called the *input*, matrix K is called the *kernel* or *filter*.
- Cross-correlation is closely related to matrix convolution (see Problem Set 5, Ex.1), which is where convolutional layers take their name from.
- In Machine Learning it is common to refer to “cross-correlation” simply as “convolution”.

6.6 CNNs: the convolution layer

You can think of matrix convolution as a moving component-wise multiplication of the input and kernel matrices.

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

=

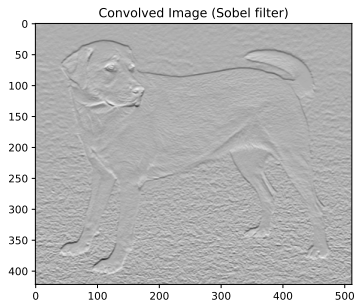
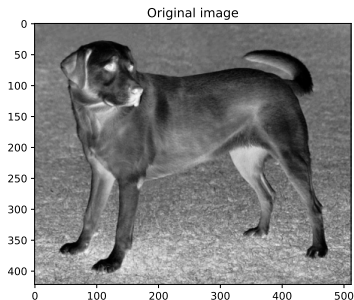
6		

$$\begin{aligned} &7 \times 1 + 4 \times 1 + 3 \times 1 + \\ &2 \times 0 + 5 \times 0 + 3 \times 0 + \\ &3 \times -1 + 3 \times -1 + 2 \times -1 \\ &= 6 \end{aligned}$$

6.6 CNNs: the convolution layer

- As an example, consider the following convolution:

$$\underbrace{\begin{pmatrix} 0.613 & 0.596 & \dots & 0.523 \\ 0.586 & 0.589 & \dots & 0.585 \\ \vdots & \vdots & \ddots & \\ 0.462 & 0.290 & \dots & 0.426 \end{pmatrix}}_{\text{Original image (grayscale)}} * \underbrace{\begin{pmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{pmatrix}}_{\text{Sobel y filter}}$$



- The convolution has preserved the input shape and has identified edges.

6.6 CNNs: the convolution layer

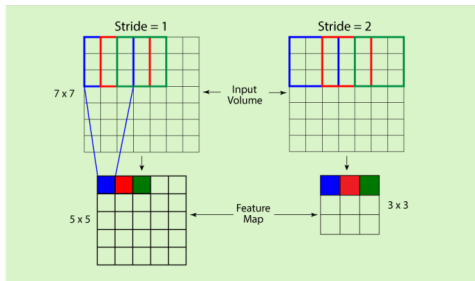
- In ML kernels are always square matrices $m \times m$, where m is a hyperparameter.
- If I has shape (n_1, n_2) and K has shape (m, m) , then the output $I \circledast K$ has shape $(n_1 - m + 1, n_2 - m + 1)$.
- To keep the same shape as the input, we can **pad** the input with p “layers” of zeros. For example, for $I \in \mathbb{R}^{3 \times 3}$ and $p = 1$:

$$I \mapsto \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & I_{1,1} & I_{1,2} & I_{1,3} & 0 \\ 0 & I_{2,1} & I_{2,2} & I_{2,3} & 0 \\ 0 & I_{3,1} & I_{3,2} & I_{3,3} & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

- The padded input has shape $(n_1 + 2p, n_2 + 2p)$,
the convolution output has shape $(n_1 + 2p - m + 1, n_2 + 2p - m + 1)$
 $\implies$ we can keep same input size if $p = (m - 1)/2$ (m must be odd).
- The padding p is an hyperparameter.

6.6 CNNs: the convolution layer

- We can compute convolution with *strides* of varying length s :



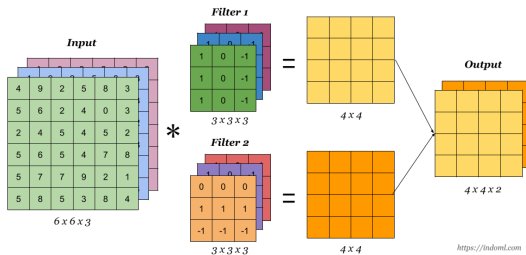
- For stride s , the output has shape $((n_1 - m)/s + 1, (n_2 - m)/s + 1)$.
- The stride s is an hyperparameter.

6.6 CNNs: the convolution layer

- Normally a convolution layer consists of $M > 1$ kernels, each with its own set of parameters, which are learned in parallel.
- If input I has shape (n_1, n_2) and we learn M kernels of shape (m, m) with padding p and stride s , then the layer's output has shape:

$$\left(\frac{n_1 + 2p - m}{s} + 1, \frac{n_2 + 2p - m}{s} + 1, M \right)$$

- RGB colored images can be represented as tensors of shape $(n_1, n_2, 3)$. Color dimensions are referred to as **channels** (e.g. B/W images have 1 channel, RGB images have 3 channels).
- Convolution for inputs with three channels is done with three-dimensional kernels.



6.6 CNNs: the detector layer

Detector layer:

- The detector layer applies a non-linear activation function $\rho : \mathbb{R} \rightarrow \mathbb{R}$ component-wise:

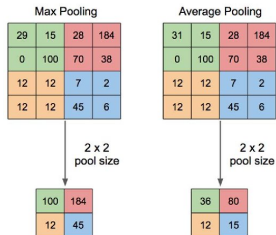
$$I_{i,j,k} \mapsto \rho(I_{i,j,k}).$$

- The output has the same shape as the input.
- The detector layer does not have trainable parameters.

6.6 CNNs: the pooling layer

Pooling layer:

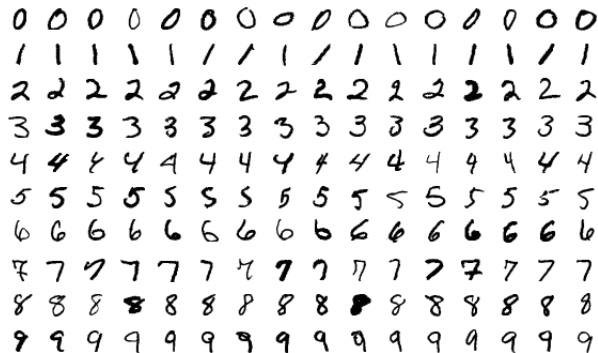
- The pooling layer applies a fixed filter of shape (m, m) , stride s and padding p (typical choices are $p = 0, m = 2, s = 2$).
- Two common types of pooling: *max pooling* and *average pooling*.



- For multi-channel inputs pooling layers act independently on each channel.
- The pooling layer typically does not have trainable parameters.

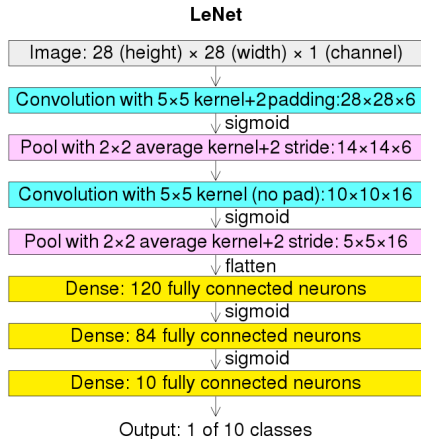
6.6 CNNs: example architecture LeNet-5

- LeNet-5 is one of the earliest convolutional neural networks. [LeCun et al., 1998].
- It was trained on the MNIST dataset, which consists of 70,000 black and white pictures of handwritten digits:



- The model classifies input images according to the digit they represent (i.e. 0, 1, ..., 9).

6.6 CNNs: example architecture LeNet-5



- **Exercise:** Check that the layers' input and output dimensions match!
- Output layer: $(z_1, \dots, z_{10}) \mapsto \left(\frac{\exp\{z_1\}}{\sum_{i=1}^{10} \exp\{z_i\}}, \dots, \frac{\exp\{z_{10}\}}{\sum_{i=1}^{10} \exp\{z_i\}} \right).$

6.6 CNNs and data priors

- What are CNNs doing in practice?

Consider an input image of shape $(3, 3)$ and a kernel K of shape $(2, 2)$:

$$I = \begin{pmatrix} I_{1,1} & I_{1,2} & I_{1,3} \\ I_{2,1} & I_{2,2} & I_{2,3} \\ I_{3,1} & I_{3,2} & I_{3,3} \end{pmatrix} \quad K = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

- The convolution $I \circledast K$ is actually a linear transformation of the **flattened input** image:

$$\begin{pmatrix} (I \circledast K)_{1,1} \\ (I \circledast K)_{1,2} \\ (I \circledast K)_{2,1} \\ (I \circledast K)_{2,2} \end{pmatrix} = \underbrace{\begin{pmatrix} a & b & 0 & c & d & 0 & 0 & 0 & 0 \\ 0 & a & b & 0 & c & d & 0 & 0 & 0 \\ 0 & 0 & 0 & a & b & 0 & c & d & 0 \\ 0 & 0 & 0 & 0 & a & b & 0 & c & d \end{pmatrix}}_{\text{doubly block-circulant matrix}} \cdot \begin{pmatrix} I_{1,1} \\ I_{1,2} \\ I_{1,3} \\ I_{2,1} \\ I_{2,2} \\ I_{2,3} \\ I_{3,1} \\ I_{3,2} \\ I_{3,3} \end{pmatrix}$$

- Most entries of the doubly block-circulant matrix are zero (*sparsity*) and the non-zero entries are repeated (*parameter sharing* or *tied weights*).

6.6 CNNs and data priors

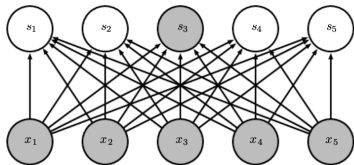
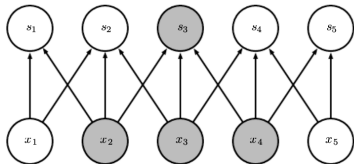
CNNs work well because they exploit structural properties or *data priors* of image data.

- **Parameter sharing**

- the same kernel parameters are used at every position in the input image
 - ⇒ learning is shared (e.g. no need to relearn how to detect edges at each position in the image)
 - ⇒ the output of the convolutional layer is translation equivariant (see Problem Set 5, Ex. 2)

- **Sparsity**

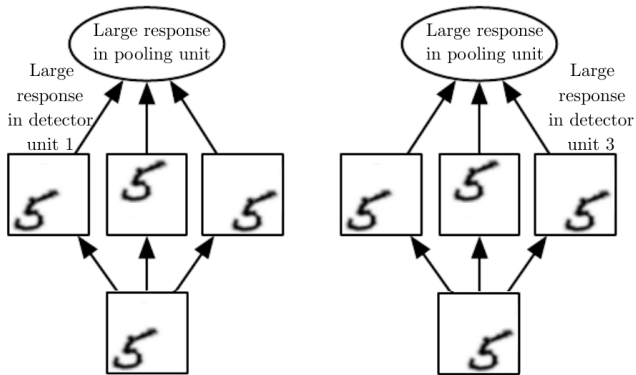
- The first convolutional layer of LeNet-5 has 150 parameters (6 kernels of shape (5, 5)), while a fully-connected FNN with identical output shape would require 614,656 weights.
 - ⇒ CNNs are computationally efficient
- Small kernels reduce the number of effective interactions among nodes
 - ⇒ lower layers extract local features (e.g. presence of edges)
 - ⇒ deeper layers extract global features (e.g. what digit is it?)



6.6 CNNs and data priors

The output of the pooling layer is invariant under small translations.

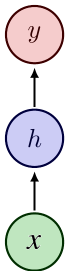
⇒ Useful for most image recognition tasks (e.g. is there a "5" in the image?).



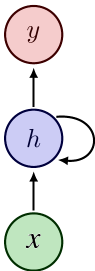
6.7 Recurrent Neural Networks

- The architecture of Recurrent Neural Networks (RNNs) is specifically meant to exploit data priors of sequential data, $(x^{(1)}, x^{(2)}, \dots, x^{(T)})$.

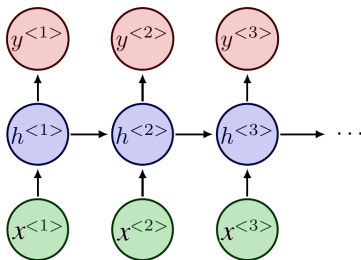
FNN



RNN



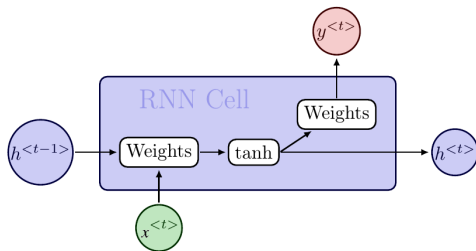
RNN unrolled



$$\begin{cases} a^{(t)} &= W \cdot h^{(t-1)} + U \cdot x^{(t)} + b \\ h^{(t)} &= \rho(a^{(t)}) \\ y^{(t)} &= V \cdot h^{(t)} + c \end{cases}$$

6.7 Recurrent Neural Networks

- Alternatively, one can think of a recurrent layer as a "cell":



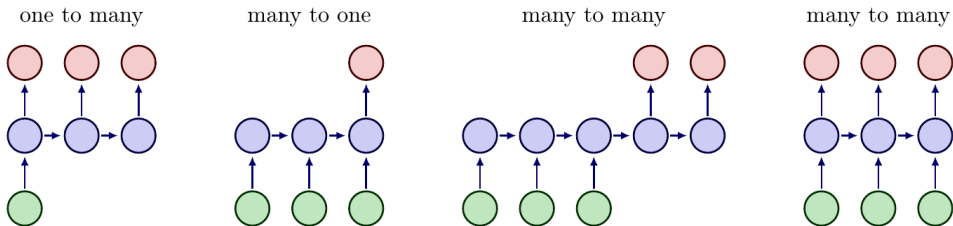
- Mathematically, the hidden states of an RNN form a dynamical system with stationary transition functions:

$$h^{<t>} = f_{\theta}(h^{<t-1>}, x^{<t>}), \quad \theta = (W, U, b).$$

- Notice that the same model parameters $\theta = (W, U, b)$ are used at each time step t , so learning is shared between time steps (***parameter sharing***)
- The network learns to use $h^{<t>}$ as a summary statistics of the past sequence of inputs and to predict $y^{<t>}$ as accurately as possible.

6.7 Recurrent Neural Networks

- Many RNN architectures are possible.
- Input and output sequence lengths may differ.



6.7 RNNs: Back-Propagation Through Time

- RNNs are trained using a variation of the back-propagation algorithm, known as ***Back-Propagation Through Time (BPTT)***.
- Consider the “many-to-many” architecture setting, with loss:

$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}^{(t)} = \sum_{t=1}^T \ell(\hat{y}^{<t>}, y^{<t>}), \quad t = 1, \dots, T$$

- We compute recursively the gradients of $\mathcal{L}$ with respect to the nodes $\hat{y}^{(t)}$ and $h^{(t)}$:

$$D_{\hat{y}^{(t)}} \mathcal{L} = D_{\hat{y}^{(t)}} \ell(\hat{y}^{(t)}, y^{(t)})$$

$$D_{h^{(T)}} \mathcal{L} = D_{\hat{y}^{(T)}} \mathcal{L} \cdot D_{h^{(T)}} \hat{y}^{(T)} = D_{\hat{y}^{(T)}} \mathcal{L} \cdot V$$

$$\begin{aligned} D_{h^{(t)}} \mathcal{L} &= D_{h^{(t+1)}} \mathcal{L} \cdot D_{h^{(t)}} h^{(t+1)} + D_{\hat{y}^{(t)}} \mathcal{L} \cdot D_{h^{(t)}} \hat{y}^{(t)} \\ &= D_{h^{(t+1)}} \mathcal{L} \cdot \text{diag} \left(\rho' \left(a^{(t+1)} \right) \right) \cdot W + D_{\hat{y}^{(t)}} \mathcal{L} \cdot V \end{aligned}$$

6.7 RNNs: Back-Propagation Through Time

- We compute the parameters' gradient through the nodes $\hat{y}^{(t)}$ and $h^{(t)}$.
- We denote by $\theta^{(t)}$ the values of the parameter θ at time t .

$$D_c \mathcal{L} = \sum_{t=1}^T D_{\hat{y}^{(t)}} \mathcal{L} \cdot D_{c^{(t)}} \hat{y}^{(t)} = \sum_{t=1}^T D_{\hat{y}^{(t)}} \mathcal{L}$$

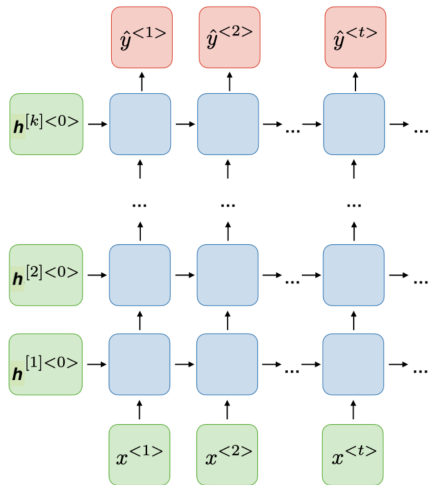
$$D_b \mathcal{L} = \sum_{t=1}^T D_{h^{(t)}} \mathcal{L} \cdot D_{b^{(t)}} h^{(t)} = \sum_{t=1}^T D_{h^{(t)}} \mathcal{L} \cdot \text{diag} \left(\rho'(a^{(t)}) \right)$$

$$D_V \mathcal{L} = \sum_{t=1}^T \sum_i D_{\hat{y}_i^{(t)}} \mathcal{L} \cdot D_{V^{(t)}} \hat{y}_i^{(t)} = \sum_{t=1}^T D_{\hat{y}^{(t)}} \mathcal{L}^T \cdot h^{(t)T}$$

$$D_W \mathcal{L} = \sum_{t=1}^T \sum_i D_{h_i^{(t)}} \mathcal{L} \cdot D_{W^{(t)}} h_i^{(t)} = \sum_{t=1}^T \text{diag} \left(\rho'(a^{(t)}) \right) \cdot D_{h^{(t)}} \mathcal{L}^T \cdot h^{(t-1)T}$$

$$D_U \mathcal{L} = \sum_{t=1}^T \sum_i D_{h_i^{(t)}} \mathcal{L} \cdot D_{U^{(t)}} h_i^{(t)} = \sum_{t=1}^T \text{diag} \left(\rho'(a^{(t)}) \right) \cdot D_{h^{(t)}} \mathcal{L}^T \cdot x^{(t)T}$$

6.7 RNNs: the vanishing/exploding gradient problem



- It's possible to compose recurrent layers and create deeper architectures.

6.7 RNNs: the vanishing/exploding gradient problem

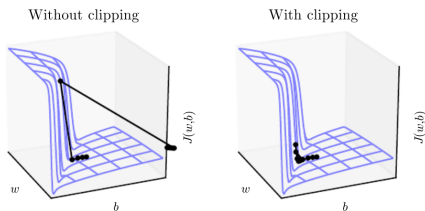
- In practice, training RNNs is difficult because of the vanishing/exploding gradient problem.
- The gradient of a shared parameter θ increases/decreases exponentially through time:

$$\begin{aligned} D_{\theta} \mathcal{L}^{(t)} &= \sum_{1 \leq k \leq t} D_{h^{(t)}} \mathcal{L}^{(t)} \cdot D_{h^{(k)}} h^{(t)} \cdot D_{\theta} h^{(k)} \\ &= \sum_{1 \leq k \leq t} D_{h^{(t)}} \mathcal{L}^{(t)} \cdot \left(\prod_{k < i \leq t} D_{h^{(i-1)}} h^{(i)} \right) \cdot D_{\theta} h^{(k)} \end{aligned}$$

- If $D_{h^{(i-1)}} h^{(i)} > 1$, then the gradient grows exponentially in $t - k$ (**exploding gradient problem**).
- If $D_{h^{(i-1)}} h^{(i)} < 1$, then the gradient vanishes exponentially in $t - k$ (**vanishing gradient problem**).

6.7 RNNs: the exploding gradient problem

- An exploding gradient is problematic because large gradient updates make learning unstable.
- Large gradient updates can “shoot” us far into suboptimal regions of the parameter space.



- The exploding gradient problem can be solved reliably by **gradient clipping**.
- At each gradient step, if the gradient is larger than a threshold K , we "clip" it to K :

$$D_{\theta} \mathcal{L} \leftarrow \begin{cases} D_{\theta} \mathcal{L} & \text{if } \|D_{\theta} \mathcal{L}\| < K \\ K \frac{D_{\theta} \mathcal{L}}{\|D_{\theta} \mathcal{L}\|} & \text{if } \|D_{\theta} \mathcal{L}\| \geq K \end{cases}$$

- The threshold K is a hyperparameter.

6.7 RNNs: the vanishing gradient problem

- The vanishing gradient problem is more complex to deal with.
- If $\rho'(\cdot)$ is bounded by γ and the largest singular value of W is strictly smaller than $1/\gamma$, then:

$$\|D_{h^{(i-1)}} h^{(i)}\|_2 = \|\text{diag}(\rho'(a^{(i)})) \cdot W\|_2 \leq \|\text{diag}(\rho'(a^{(i)}))\|_2 \cdot \|W\|_2 \leq \eta < 1$$

Therefore:

$$\|D_{\theta} \mathcal{L}^{(t)}\|_2 \leq \|D_{h^{(t)}} \mathcal{L}^{(t)}\|_2 \sum_{1 \leq k \leq t} \eta^{t-k} \|D_{\theta} h^{(k)}\|_2$$

- Long-term term contributions ($k \ll t$) vanish exponentially fast in $t - k$
 $\implies$ learning long-term dependencies takes more iterations and the gradient is submerged by short-term noise.
- This problem is typical of tied-weights.
- The vanishing gradient problem can be solved by designing more advanced recurrent units (GRUs and LSTM cells) or changing the architecture (e.g. by adding skip connections).

6.7 RNNs: the Gated Recurrent Unit (GRU)

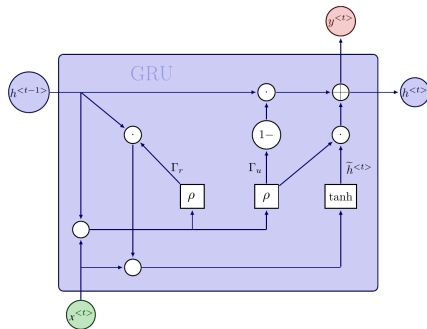
$$\Gamma_r = \rho \left(W^r \cdot h^{\langle t-1 \rangle} + U^r x^{\langle t \rangle} + b_r \right)$$

$$\Gamma_u = \rho \left(W^u \cdot h^{\langle t-1 \rangle} + U^u x^{\langle t \rangle} + b_u \right)$$

$$\tilde{h}^{\langle t \rangle} = \tanh \left(W^{\text{in}} \cdot \Gamma_r \odot h^{\langle t-1 \rangle} + U^{\text{in}} x^{\langle t \rangle} + b \right)$$

$$h^{\langle t \rangle} = (1 - \Gamma_u) \odot h^{\langle t-1 \rangle} + \Gamma_u \odot \tilde{h}^{\langle t \rangle}$$

$$y^{\langle t \rangle} = W^{\text{out}} \cdot h^{\langle t \rangle}$$



- The **update gate** Γ_u chooses to propagate either $h^{\langle t-1 \rangle}$ or a candidate state $\tilde{h}^{\langle t \rangle}$.
- If $\Gamma_u = 0$, then h is propagated identically at all future times
 $\implies$ no vanishing gradient $\implies$ long-term information is accumulated.
- $\tilde{h}^{\langle t \rangle}$ can use the **reset gate** Γ_r to selectively ignore values of $h^{\langle t-1 \rangle}$ (long-term information can be deleted).
- By learning the optimal gate parameters, **the model can learn how to accumulate/forget past information dynamically** (i.e. as a function of input $x^{\langle t \rangle}$).

6.7 RNNs: the Long Short-Term Memory (LSTM) Unit

$$\Gamma_f = \rho \left(W^f \cdot h^{\langle t-1 \rangle} + U^f \cdot x^{\langle t \rangle} + b_f \right)$$

$$\Gamma_i = \rho \left(W^i \cdot h^{\langle t-1 \rangle} + U^i \cdot x^{\langle t \rangle} + b_i \right)$$

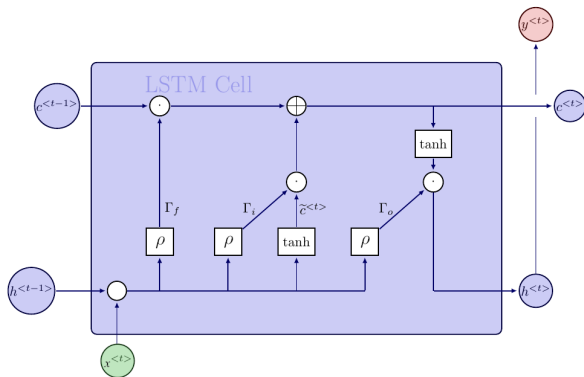
$$\Gamma_o = \rho \left(W^o \cdot h^{\langle t-1 \rangle} + U^o \cdot x^{\langle t \rangle} + b_o \right)$$

$$\tilde{c}^{\langle t \rangle} = \tanh \left(W^c h^{\langle t-1 \rangle} + U^c x^{\langle t \rangle} + b_c \right)$$

$$c^{\langle t \rangle} = \Gamma_f \odot c^{\langle t-1 \rangle} + \Gamma_i \odot \tilde{c}^{\langle t \rangle}$$

$$h^{\langle t \rangle} = \Gamma_o \odot \tanh(c^{\langle t \rangle})$$

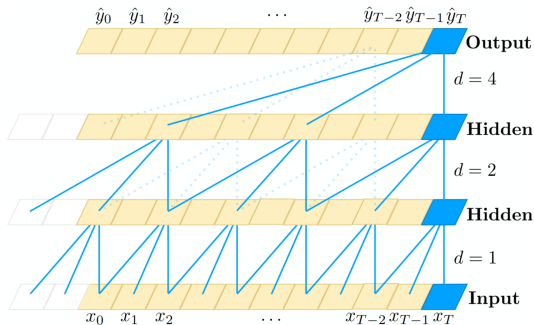
$$y^{\langle t \rangle} = W^{\text{out}} \cdot h^{\langle t \rangle}$$



- The **memory cell** c propagates information through time separately from the output h (unlike GRU).
- The **output gate** Γ_o decides how this information gets used for output.
- The **forget gate** Γ_f and **input gate** Γ_i replace the single update gate of GRUs and can model more complex dependencies.

6.8 Temporal Convolutional Networks

- First introduced by [Waibel et al., 1989], but popularized only in 2018 by [Bai et al., 2018], when more computational power became available.
- **Main idea:** stack **causal**, **dilated** convolutional layers, $(x \circledast K)_i = \sum_{n=0}^{m-1} x_{i-d \cdot n} K_n$.

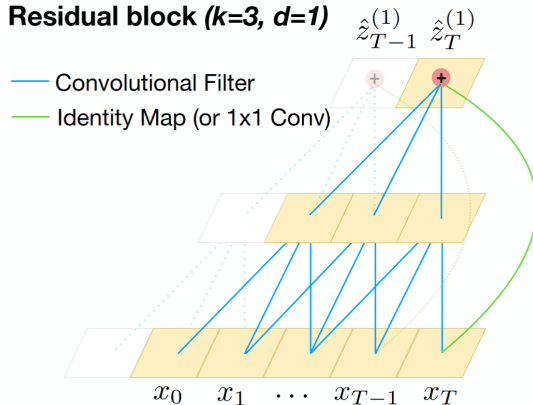


- **Causal:** output y_t is a function only of the past, $(x_s, s \leq t)$.
- **Dilated:** $d = O(2^i)$ for i -th layer $\Rightarrow y_t$ is a function of an exponentially long history.

TCNs: residual connection

- **Problem:** Long-memory tasks require **very deep** TCNs $\Rightarrow$ **unstable** training.
- **Solution:** Stabilize learning using **residual connections**.

Residual block ($k=3, d=1$)



- The output is given by $z = x + F(x)$, where F is the convolutional layer.
 $\Rightarrow F$ only needs to **learn "residuals" of the identity transformation**.
- If shape mismatch between x and z , use a suitable 1×1 convolution.

TCNs: performance

In practice TCNs perform very well and require less compute/memory.

Sequence Modeling Task	Model Size ($\approx$)	Models			
		LSTM	GRU	RNN	TCN
Seq. MNIST (accuracy ^h)	70K	87.2	96.2	21.5	99.0
Permuted MNIST (accuracy)	70K	85.7	87.3	25.3	97.2
Adding problem $T=600$ (loss ^ℓ)	70K	0.164	5.3e-5	0.177	5.8e-5
Copy memory $T=1000$ (loss)	16K	0.0204	0.0197	0.0202	3.5e-5
Music JSB Chorales (loss)	300K	8.45	8.43	8.91	8.10
Music Nottingham (loss)	1M	3.29	3.46	4.05	3.07
Word-level PTB (perplexity ^ℓ)	13M	78.93	92.48	114.50	88.68
Word-level Wiki-103 (perplexity)	-	48.4	-	-	45.19
Word-level LAMBADA (perplexity)	-	4186	-	14725	1279
Char-level PTB (bpc ^ℓ)	3M	1.36	1.37	1.48	1.31
Char-level text8 (bpc)	5M	1.50	1.53	1.69	1.45

Advantages:

- **Parallelism**: the input sequence can be processed in parallel (not sequentially, as in RNNs).
- **Lightweight model**: no need to store intermediate cells/gates values.
- **Flexible history length**: adjustable in many ways (model depth, dilation, kernel size).
- **No exploding/vanishing gradient**: backpropagation is not in the temporal direction.

Disadvantages:

- **No recurrent hidden state**: no sequence summarization.
- **Poor transfer learning**: effective history length is tied to the specific architecture.

References I

-  Bai, S., Kolter, J. Z., and Koltun, V. (2018).
An empirical evaluation of generic convolutional and recurrent networks for sequence modeling.
arXiv preprint arXiv:1803.01271.
-  Herberg, E. (2023).
Neural network architectures.
arXiv preprint arXiv:2304.05133.
-  LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998).
Gradient-based learning applied to document recognition.
Proceedings of the IEEE, 86(11):2278–2324.
-  Waibel, A., Hanazawa, T., Hinton, G., Shikano, K., and Lang, K. J. (1989).
Phoneme recognition using time-delay neural networks.
IEEE Transactions on Acoustics, Speech, and Signal Processing, 37(3):328–339.